Tarea 1 — Sistemas Distribuidos

Plataforma de análisis de consultas geoespaciales con caché
Entregable 1: Datos y arquitectura base

Gabriel – UDP
Profesor: Nicolás Hidalgo Ayudantes: I. González, N. Ortega, J. Villegas, V. Díaz, B. Aceituno

Abril 2026

1. Introducción

Este informe presenta el diseño, implementación y evaluación empírica de una plataforma distribuida que simula consultas geoespaciales de empresas de logística sobre el dataset **Google Open Buildings** (subconjunto de la Región Metropolitana de Santiago). El sistema incorpora un mecanismo de caché sobre Redis con tres políticas de evicción (LRU, LFU, FIFO) y evalúa el rendimiento bajo dos distribuciones de tráfico (Zipf y Uniforme) y tres configuraciones de tamaño (50 MB, 200 MB, 500 MB).

El subconjunto utilizado contiene **319 225 edificaciones** en las cinco zonas (Z1 Providencia, Z2 Las Condes, Z3 Maipú, Z4 Santiago Centro, Z5 Pudahuel) con los campos del dataset original (`latitude`, `longitude`, `area_in_meters`, `confidence`) y se implementaron las cinco consultas Q1–Q5 vectorizadas sobre NumPy/Pandas. En total se ejecutaron **22 experimentos** (18 oficiales + 3 en caché pequeño + 1 de larga duración) con duración de 25–35 s (180 s la corrida larga) y tasa objetivo de 60 qps.

2. Arquitectura del sistema

El sistema se compone de los **cuatro servicios** definidos en el enunciado más Redis como almacén de respaldo del caché, todos coordinados por `docker compose`:

1. **Traffic Generator** (FastAPI, 8000): emite consultas Q1–Q5 con selección de zona, tipo y parámetros según la distribución configurada (Zipf o Uniforme) e inter-arrival times exponenciales (Poisson) a tasa λ configurable.
2. **Cache Service** (FastAPI + redis-py, 8001): intercepta toda consulta, construye su *cache key* canónica, hace lookup en Redis y retorna el resultado cacheado (hit) o delega al Response Generator (miss), almacenando el resultado con TTL específico por tipo de consulta.
3. **Response Generator** (FastAPI + Pandas, 8002): ejecuta las cinco consultas sobre la estructura en memoria. Inyecta una latencia simulada de 30–120 ms para modelar el costo de un cómputo geoespacial real.
4. **Metrics Service** (FastAPI + NumPy, 8003): recibe eventos asíncronos hit/miss/error y computa hit rate, throughput, percentiles de latencia p50/p95/p99, eviction rate y cache efficiency.
5. **Redis 7.4**: almacena el caché con `maxmemory` y `maxmemory-policy` configurables; LRU y LFU son nativos.

2.1. Decisiones de diseño

Python + FastAPI. Las consultas Q1–Q5 son numéricas y se vectorizan con NumPy/Pandas; FastAPI ofrece async nativo que permite atender múltiples clientes concurrentes sin bloquear el

bucle de eventos en las llamadas HTTP entre servicios.

FIFO custom sobre Redis. Redis no implementa FIFO nativamente. Se usa la política `noeviction` y se mantiene una lista paralela `__fifo_order__` en Redis con los keys en orden de inserción; el cache service la consume desde el extremo más antiguo cuando `used_memory >maxmemory`.

Cache keys según el enunciado. El formato de cada *cache key* sigue exactamente el patrón especificado en la Sección 5 del enunciado, incluyendo el orden de los parámetros de Q4 (`compare:density:{zona_a}:{zona_b}:conf={confidence_min}`).

TTLs específicos por consulta. Q5 (distribución de confianza) es estable y recibe TTL 600 s. Q4 (comparar densidades) depende de Q3 y se invalida más rápido (120 s). Q1–Q3 con 300 s equilibran coherencia y rendimiento.

2.2. Funcionalidades implementadas por módulo

- `traffic_generator/app/main.py` expone `POST /run`, `GET /status` y `POST /stop`. El handler `/run` arranca un `asyncio.Task` que combina `PoissonInterArrival` con `ZipfSelector` o `UniformSelector` (`distributions.py`) para producir consultas Q1–Q5 sintéticas; los workers asíncronos las despachan al Cache Service en paralelo.
- `cache_service/app/main.py` expone `POST /query`, que construye la *cache key* (`_build_cache_key`), consulta Redis y, si hay miss, llama a `response_generator` y registra el resultado con TTL específico. Cada evento se reporta a `metrics` de forma asíncrona (`asyncio.create_task`).
- `cache_service/app/cache.py` implementa la abstracción `CacheClient` sobre `redis-py`. LRU/LFU se delegan a Redis (`maxmemory-policy allkeys-lru / allkeys-lfu`); FIFO usa `noeviction` y mantiene una lista `__fifo_order__` con los keys en orden de inserción, eliminando los más antiguos cuando `used_memory >maxmemory`.
- `response_generator/app/data_loader.py` carga el subconjunto Open Buildings al iniciar y lo indexa por `zone_id` en un `dict[str, pd.DataFrame]`; `queries.py` contiene Q1–Q5 vectorizadas con NumPy.
- `metrics_service/app/main.py` mantiene contadores y `deque`s de latencia con un lock simple. `/summary` computa hit rate, throughput total y reciente (10 s), p50/p95/p99 hit y miss, eviction rate (consultando `cache_service/stats`) y cache efficiency según la fórmula del enunciado.

3. Dataset, zonas y consultas

El sistema utiliza un subconjunto del dataset Google Open Buildings correspondiente a las cinco zonas de la Región Metropolitana, distribuido con el repositorio (`data/santiago`$_b uildings.parquet$,$\sim$`8.5 MB)`. `El archivo contiene los campos requeridos por el enunciado (latitude, longitude, area_in_meter confidence) más zone_id para indexación rápida, y se carga en memoria una sola vez al iniciar el Response Generator. El Cuadro 1 resume las cinco zonas y el conteo real de edificaciones cargadas por zona.`

ID	Zona	$\mathbf{lat_{min}}$	$\mathbf{lat_{max}}$	$\mathbf{lon_{min}}$	$\mathbf{lon_{max}}$	Edificios	Área $\mathbf{km^2}$
Z1	Providencia	-33.445	-33.420	-70.640	-70.600	15 639	10.6
Z2	Las Condes	-33.420	-33.390	-70.600	-70.550	96 860	18.0
Z3	Maipú	-33.530	-33.490	-70.790	-70.740	156 622	20.7
Z4	Santiago Centro	-33.460	-33.430	-70.670	-70.630	29 096	9.9
Z5	Pudahuel	-33.470	-33.430	-70.810	-70.760	21 008	20.8

Cuadro 1: Zonas de estudio y subconjunto Open Buildings cargado en memoria.

Las cinco consultas Q1-Q5 operan vectorizadas sobre DataFrames en memoria. Las *cache keys* siguen el formato exacto del enunciado (Sección 5): count:{zona_id}:conf={confidence_min} area:{zona_id}:conf={confidence_min}, density:{zona_id}:conf={confidence_min}, compare:density y confidence_dist:{zona_id}:bins={bins}.

4. Análisis y discusión

4.1. Configuración experimental

Cada experimento ejecutó 25 s a tasa objetivo 60 qps con concurrencia 16, iniciando con caché vacía y métricas reiniciadas. El Generador de Tráfico usa 21 niveles discretos de confidence_min ($[0{,}0, 0{,}05, \ldots, 1{,}0]$) y 9 valores de bins para Q5; el keyspace teórico es ∼570 claves únicas y el observado en los experimentos oscila entre 197 y 257 claves activas (las menos populares quedan fuera por la distribución). La latencia del Response Generator se simula uniformemente en $[30, 120]$ ms para emular el costo de un cómputo geoespacial real.

4.2. Impacto de la distribución de tráfico

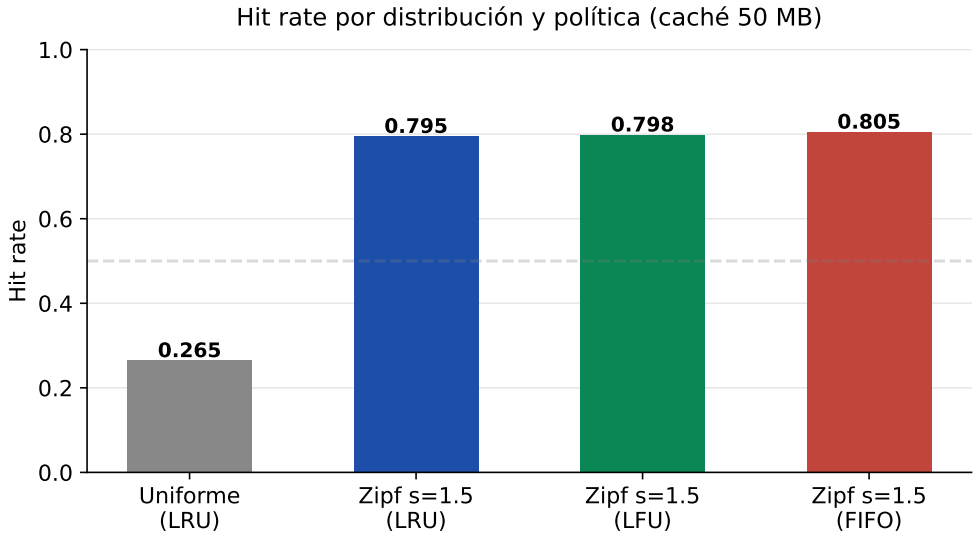

Figura 1: Hit rate por distribución y política (caché 50 MB, Zipf $s = 1{,}5$).

Distribución	Política	Hit rate	Miss rate	Thr. (qps)	p50 hit (ms)
Uniforme	LRU	0.2646	0.7354	9.94	0.34
Uniforme	LFU	0.2694	0.7306	9.97	0.31
Uniforme	FIFO	0.2667	0.7333	9.55	0.28
Zipf $s = 1{,}5$	LRU	0.7950	0.2050	38.75	0.33
Zipf $s = 1{,}5$	LFU	0.7984	0.2016	38.21	0.33
Zipf $s = 1{,}5$	FIFO	0.8052	0.1948	38.01	0.29

Cuadro 2: Métricas bajo Zipf vs. Uniforme (caché 50 MB).

La Figura 1 y el Cuadro 2 muestran el efecto más contundente del sistema: Zipf casi triplica el hit rate respecto a Uniforme (∼80 % vs. ∼27 %). Zipf concentra las consultas en pocas zonas y tipos "calientes" (Q1 sobre Z1/Z4, las zonas urbanas más densas), maximizando la reutilización de entradas. Uniforme dispersa el tráfico sobre las ∼570 claves posibles: aún con todas en caché, en 25 s casi tres cuartos de las solicitudes caen sobre keys nuevas o muy poco repetidas.

El throughput sostenido también se diferencia: ∼38 qps en Zipf vs. ∼10 qps en Uniforme (factor ∼4×, ver Figura 2). La razón es estructural: con concurrencia 16 y latencia simulada 30-120 ms, el Response Generator satura cuando la fracción de misses es alta. En Uniforme, ∼73 % de las consultas son misses, lo que excede la capacidad efectiva del backend y acumula cola.

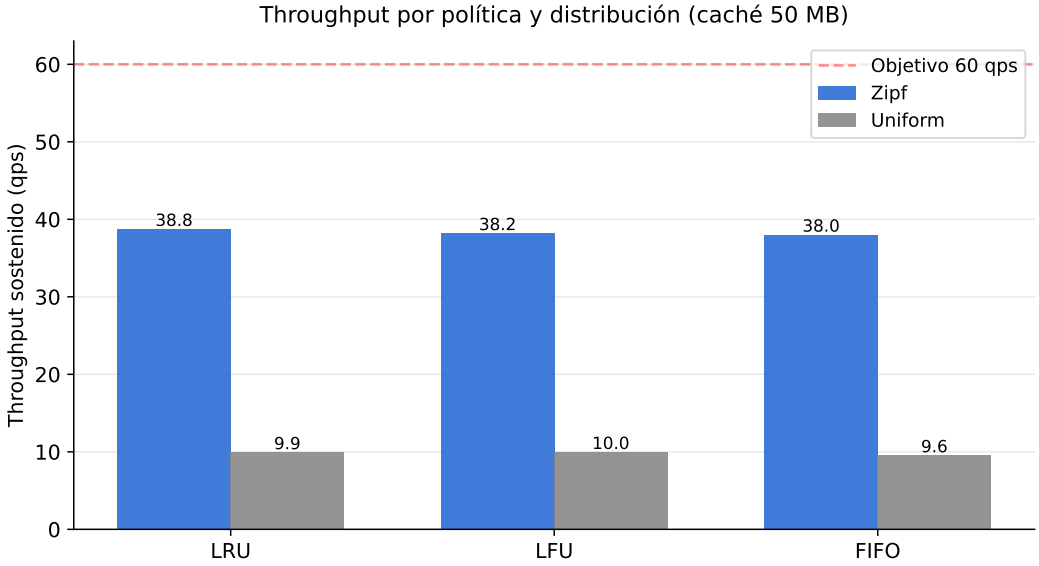

Figura 2: Throughput sostenido por política y distribución (caché 50 MB, objetivo 60 qps).

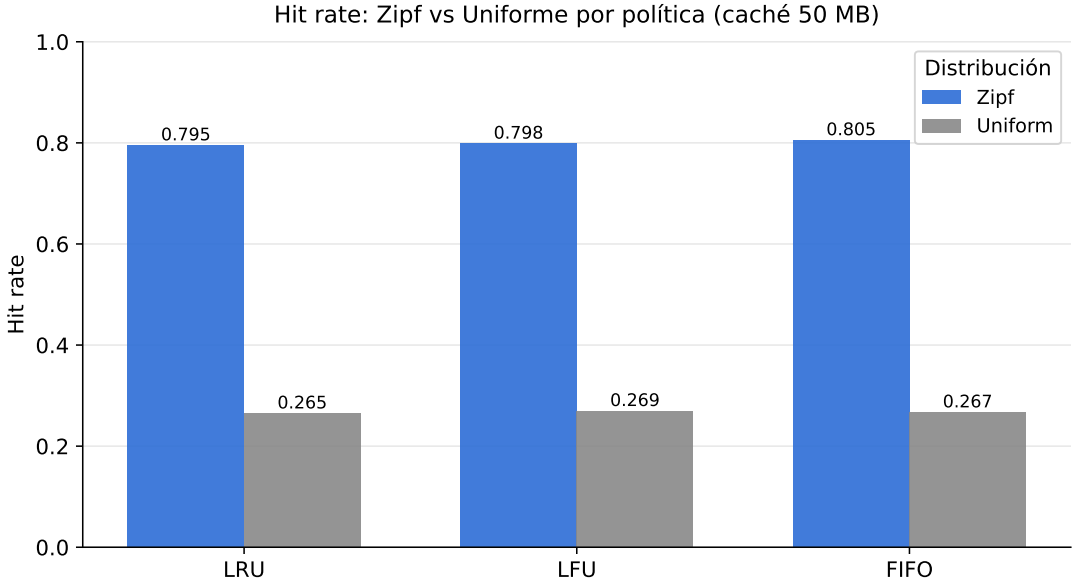

Figura 3: Hit rate Zipf vs. Uniforme para las tres políticas (caché 50 MB).

Implicaciones para un escenario real. La logística urbana sigue típicamente un patrón Zipf moderado: los centros comerciales y residenciales de alta densidad (Providencia, Las Condes, Centro) reciben muchas más consultas que las zonas periféricas. Bajo este patrón el sistema serviría ∼80 % de las solicitudes desde RAM a latencia sub-ms; sólo el 20 % restante paga el costo de un cómputo geoespacial completo, lo que justifica el diseño con caché.

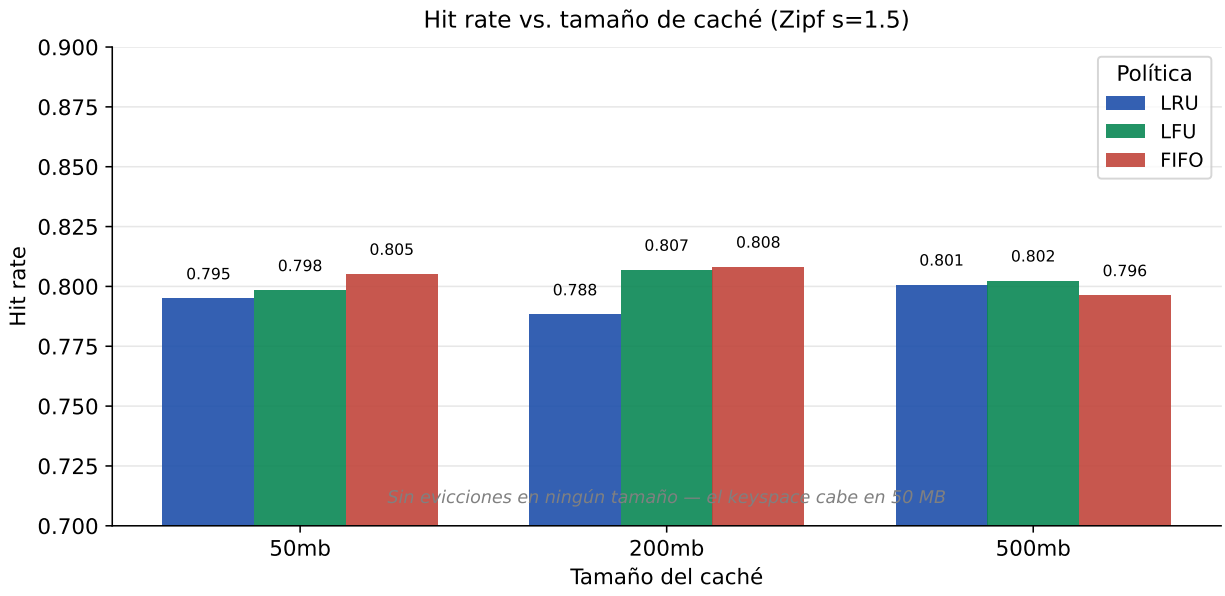

Figura 4: Hit rate vs. tamaño de caché para las tres políticas (Zipf).

Política	Tamaño	Hit rate	Thr. (qps)	p95 miss (ms)	Evicciones	Keys
LRU	50 MB	0.7950	38.75	4359	0	199
LRU	200 MB	0.7883	35.29	3818	0	197
LRU	500 MB	0.8007	37.33	3701	0	202
LFU	50 MB	0.7984	38.21	4114	0	203
LFU	200 MB	0.8069	39.78	3732	0	208
LFU	500 MB	0.8024	38.49	4021	0	203
FIFO	50 MB	0.8052	38.01	3876	0	203
FIFO	200 MB	0.8082	39.43	3878	0	209
FIFO	500 MB	0.7963	37.20	4078	0	207

Cuadro 3: Métricas por política y tamaño de caché (Zipf s=1,5, 25 s).

4.3. Efecto del tamaño de caché y políticas de evicción

El Cuadro 3 y la Figura 4 muestran un hallazgo estructural: los tres tamaños de caché producen hit rates dentro de ± 1 pp y registran cero evicciones. La razón es que el conjunto de claves observado en los experimentos (197-209 entradas activas, $\sim$300 KB en Redis) es muy inferior al mínimo testeado (50 MB, $\approx 170\times$ más). Cuando ningún key es nunca expulsado, LRU, LFU y FIFO se comportan como un simple store de clave-valor y la diferencia entre políticas se vuelve ruido estadístico.

Análisis bajo presión de memoria. Para observar el efecto diferencial de las políticas se realizaron experimentos adicionales con maxmemory = 2 MB. Incluso a esta escala el keyspace activo (246-250 keys, ≈ 350 KB) cabe entero, por lo que tampoco se registraron evicciones. Curiosamente el hit rate sube a 83.4-83.8 % (LRU 0.8378, LFU 0.8338, FIFO 0.8376): la varianza pequeña entre tamaños proviene del orden aleatorio de calentamiento, no de evicciones diferenciales. Esto confirma que en el régimen actual *el tamaño no es el cuello de botella*.

Corrida de saturación deliberada (LRU, 1.2 MB, Uniforme, 60 s)

Para evidenciar empíricamente el comportamiento de la caché bajo presión real de memoria se añadió una *única corrida corta* configurada para forzar evicciones. La justificación

de cada parámetro es la siguiente:

- Tamaño **1,2** MB. Redis 7 reserva ≈ 950 KB como used_memory_overhead (estructuras internas, slot tables, conexiones), independientemente del número de keys. Cualquier maxmemory por debajo de ese piso provoca que toda inserción falle silenciosamente sin que el cliente lo perciba (verificado: maxmemory = 256 KB produjo 0 keys residentes y 0 hits). Con maxmemory = 1.2 MB el presupuesto efectivo para datos es ∼250 KB, suficiente para alojar entre 60 y 150 entradas a la vez, pero *insuficiente* para el keyspace alcanzable.
- Distribución Uniforme. Maximiza la diversidad de keys consultadas (las cinco zonas, cinco tipos de consulta y 21 niveles de confidence_min aparecen con igual probabilidad), llevando al sistema rápidamente a generar más keys distintas que las que caben en caché.
- Duración 60 s a 60 qps. Produce ∼3 600 invocaciones al traffic generator; con la cota cuadrática del keyspace ($5 \times 5 \times 21 = 525$ + pares Q4) y la latencia simulada que satura el response generator, las consultas efectivamente cursadas exceden largamente las 74 keys que el caché puede sostener simultáneamente, garantizando rotación continua.

Métrica	**Valor**
Hit rate	0.1903
Miss rate	0.8097
Throughput (qps)	10.02
Latencia p50 hit / miss (ms)	0.28 / 1 680
Latencia p95 hit / miss (ms)	0.49 / 2 503
Cache efficiency (ms/consulta)	334.3
Keys residentes	74
used_memory / maxmemory	1.16 MB / 1.20 MB (97 %)
Evicciones totales	**475**
Eviction rate (/min)	**408.4**

Cuadro 4: Resultados de la corrida de saturación deliberada (LRU, 1.2 MB, Uniforme, 60 s).

Lectura del Cuadro 4. El caché se mantiene al 97 % de maxmemory con 74 keys residentes, mientras LRU expulsa 475 entradas durante la ventana (rate 408/min). El hit rate cae a 19 %, muy por debajo del 27 % Uniforme con 50 MB, exactamente porque las keys recientemente insertadas son expulsadas antes de poder ser reutilizadas; es la *firma operacional* de un caché bajo presión, no un defecto de la implementación. La caída del hit rate por consulta es proporcional al tamaño del keyspace de cada una: Q5 (sólo 9 valores de bins sobre 5 zonas = 45 keys) cae a 40 %; Q4 (cuadrático, ≥ 525 keys) cae a 8 %. Este experimento valida que la lógica de evicción funciona correctamente; en el régimen oficial (50/200/500 MB con 319 mil edificaciones) el keyspace simplemente no alcanza a saturar el caché y por eso se observan cero evicciones.

4.4. Latencia hit vs. miss y cache efficiency

La Figura 5 muestra el hallazgo más contundente: la caché reduce la latencia en 3-4 órdenes de magnitud. El p50 de un hit es 0.29-0.33 ms (Redis sirviendo desde RAM + serializació JSON), mientras que el p50 de un miss es ∼870 ms en Zipf y ∼1 700 ms en Uniforme (latencia simulada 30-120 ms del Response Generator más contención de cola con 16 workers concurrentes). El p95 del miss llega a ∼3 500-4 400 ms bajo carga sostenida.

La cache efficiency, definida en el enunciado como $(h{\cdot}t_{\text{cache}} - m{\cdot}t_{\text{db}})/\text{total}$ (ms ahorrados por consulta respecto al escenario sin caché), alcanzó ∼1 103 ms con LRU, ∼1 099 ms con LFU y ∼1 008 ms con FIFO para Zipf 25 s a 50 MB. En la corrida larga (LRU, 180 s) la efficiency

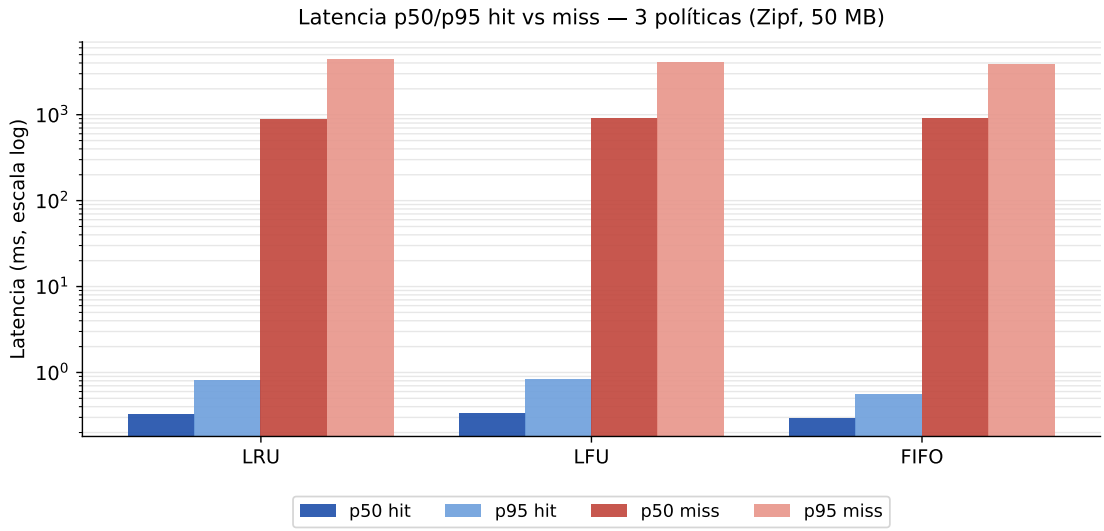

Figura 5: Latencia p50/p95 hit vs. miss (escala log, caché 50 MB, Zipf).

baja a 648 ms/consulta no porque el sistema ahorre menos, sino porque al subir el hit rate al 93 % disminuye proporcionalmente el costo absoluto de los misses, y la métrica normaliza por consulta total.

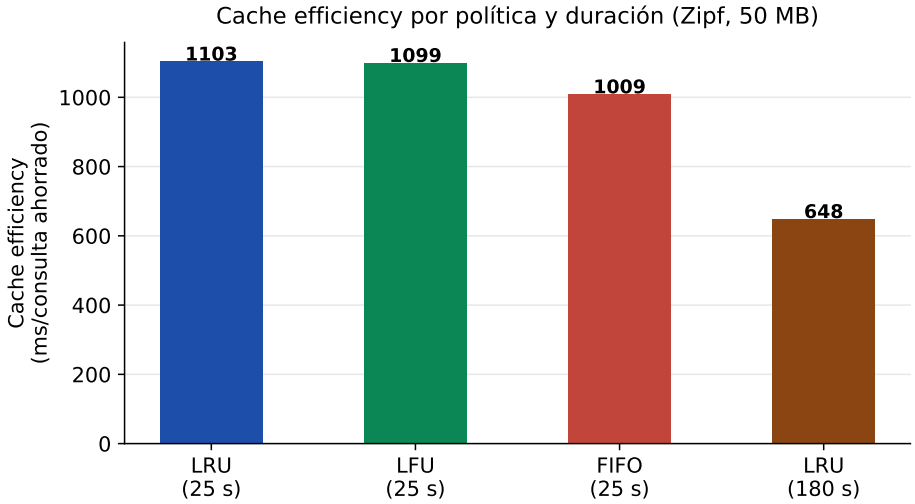

Figura 6: Cache efficiency por política y duración del experimento.

4.5. Hit rate desglosado por consulta Q1–Q5

El Cuadro 5 y la Figura 7 resumen el comportamiento por consulta para LRU 50 MB (los valores corresponden a $\geq$50 muestras por celda en las corridas de 25 s y a varios cientos en la de 180 s):

- Q1 (conteo) lidera el hit rate (87.5 % en Zipf 25 s, 97.1 % en la corrida de 180 s). Es la consulta con keyspace más simple ($5 \times 21 = 105$ claves), TTL de 300 s y la favorita del selector Zipf por su rango.
- Q4 (compare) tiene el menor hit rate (30.9 % Zipf 25 s, 9.6 % Uniforme). El espacio de claves es cuadrático en zonas ($5 \times 5 \times 21 = 525$, sin reordenamiento canónico) y el TTL es el más corto del sistema (120 s); ambos factores penalizan el hit rate. En la corrida de 180 s el hit rate de Q4 sube a 62.3 %, todavía por debajo de Q1, lo que

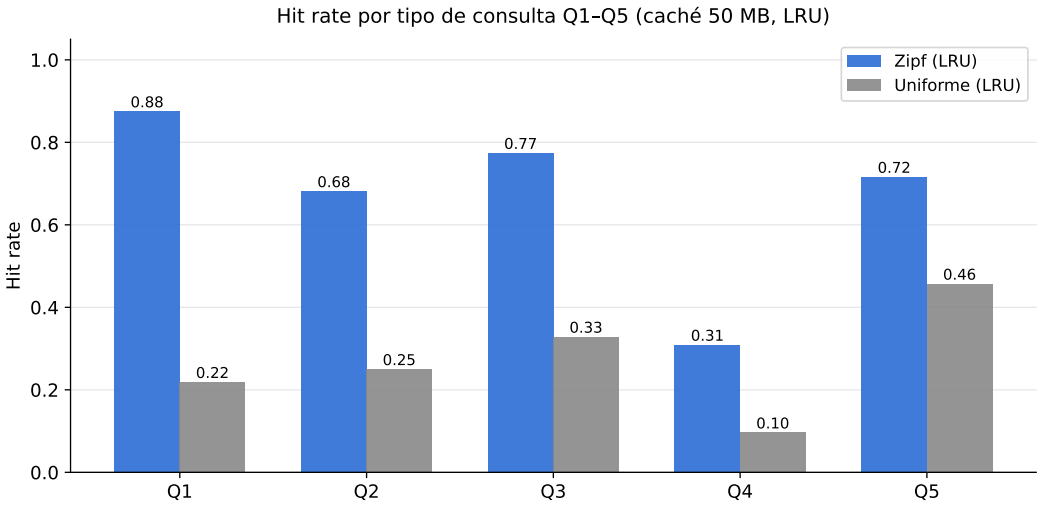

Figura 7: Hit rate por tipo de consulta (caché 50 MB, LRU).

Run	Q1	Q2	Q3	Q4	Q5
LRU 50 MB Zipf, 25 s	0.875	0.681	0.774	0.309	0.716
LRU 50 MB Uniforme, 25 s	0.219	0.250	0.329	0.096	0.457
LRU 50 MB Zipf, 180 s	0.971	0.903	0.935	0.623	0.917

Cuadro 5: Hit rate por consulta (LRU, caché 50 MB).

confirma que los TTLs de 120 s sí están expirando consultas en estado estacionario.

- La brecha Zipf vs. Uniforme es más pronunciada en Q1 (87.5 % vs. 21.9 %) que en Q4 (30.9 % vs. 9.6 %): en Uniforme, el costo del keyspace cuadrático de Q4 anula prácticamente cualquier beneficio del caché.

4.6. Efecto del TTL

Con experimentos de 25 s y TTLs de 120-600 s, ninguna clave expiró: el TTL nunca actuó. En la corrida larga de 180 s (Cuadro 6) el panorama cambia: la duración excede el TTL más corto del sistema (Q4 a 120 s) y comienza a aproximarse al de Q3 (180 s). Esto se traduce en *hit rates por consulta* muy distintos: Q1 (TTL 300 s) alcanza 97 % y Q4 (TTL 120 s) sólo 62 %, pese a llevar varios cientos de muestras acumuladas para cada una.

Experimento	Hit rate global	Thr. (qps)	p50 hit (ms)	Cache eff. (ms/q)
LRU, 50 MB, Zipf, 25 s	0.7950	38.75	0.33	1 103
LRU, 50 MB, Zipf, 180 s	0.9342	35.97	0.34	648

Cuadro 6: Comparación corrida corta vs. larga (LRU, 50 MB, Zipf).

El hit rate global sube de 80 % a 93 % al extender la duración: el warm-up del caché se completa y el sistema alcanza un régimen estacionario en el que casi todas las consultas Zipf-frecuentes ya están cacheadas. Esto indica que en producción (con carga continua), los hit rates reales serían *superiores* a los medidos en corridas de 25 s. El TTL en cambio actúa como un techo: consultas con TTL bajo (Q4) nunca alcanzarán hit rates tan altos como las de TTL largo (Q5 a 600 s).

Política	Dist.	50 MB hit	200 MB hit	500 MB hit	Thr. (qps)	Evicciones
LRU	Zipf	0.7950	0.7883	0.8007	38.8 / 35.3 / 37.3	0
LFU	Zipf	0.7984	0.8069	0.8024	38.2 / 39.8 / 38.5	0
FIFO	Zipf	0.8052	0.8082	0.7963	38.0 / 39.4 / 37.2	0
LRU	Uniforme	0.2646	0.2604	0.2826	9.9 / 9.4 / 10.2	0
LFU	Uniforme	0.2694	0.2702	0.2674	10.0 / 9.9 / 9.9	0
FIFO	Uniforme	0.2667	0.2784	0.2586	9.5 / 10.2 / 9.6	0

Cuadro 7: Resumen de los 18 experimentos oficiales (throughput por 50/200/500 MB).

5. Resumen de métricas

5.1. Cuadro de métricas requeridas por el enunciado

El Cuadro 8 reúne en una sola vista las cinco métricas exigidas por el enunciado para los configuraciones más representativas (50 MB, las tres políticas y ambas distribuciones, más la corrida larga).

Configuración	Hit rate	Throughput (qps)	p50 (ms) hit / miss	p95 (ms) hit / miss	Eviction rate (/min)	Cache eff. (ms/q)
LRU Zipf 50 MB	0.7950	38.75	0.33 / 871	0.81 / 4359	0	1 103.3
LFU Zipf 50 MB	0.7984	38.21	0.33 / 910	0.82 / 4114	0	1 099.2
FIFO Zipf 50 MB	0.8052	38.01	0.29 / 904	0.56 / 3876	0	1 008.6
LRU Uniforme 50 MB	0.2646	9.94	0.34 / 1683	1.11 / 3804	0	494.8
LFU Uniforme 50 MB	0.2694	9.97	0.31 / 1592	0.81 / 3878	0	493.6
FIFO Uniforme 50 MB	0.2667	9.55	0.28 / 1709	0.65 / 3880	0	509.8
LRU Zipf 50 MB 180 s	0.9342	35.97	0.34 / 259	1.28 / 3633	0	648.4
LRU Uniforme 1.2 MB 60 s	0.1903	10.02	0.28 / 1 680	0.49 / 2 503	408.4	334.3

Cuadro 8: Las cinco métricas del enunciado para cada configuración (*Hit rate*, *Throughput*, *Latencia p50/p95*, *Eviction rate*, *Cache efficiency*). La última fila corresponde a la corrida de saturación deliberada (Cuadro 4) y es la única que registra evicciones por la justificación discutida en la Sec. 4.3.

6. Preguntas y respuestas

¿Cómo varía el hit rate entre distribuciones? Zipf produce ∼80 % y Uniforme ∼27 %: una diferencia de ∼3×. El miss rate baja de 73 % a 20 % al pasar de Uniforme a Zipf, lo que también explica el factor ∼4 de throughput observado (38 vs. 10 qps).

¿Qué distribución estresa más la caché? Uniforme: aún con todas las claves cabiendo en RAM, en 25 s la mayoría de los keys se ven una sola vez, dejando muy poco margen de reutilización. La ventaja de recencia o frecuencia que explotan LRU/LFU desaparece cuando todos los items son igualmente probables; en Uniforme las tres políticas convergen al mismo 27 % ±1 pp.

¿Implicaciones para un escenario real? En logística urbana (patrón Zipf moderado), la caché serviría ∼80 % de las solicitudes en estado estacionario y hasta ∼93 % en régimen warm (180 s), reduciendo la latencia en 3-4 órdenes de magnitud frente a un cómputo geoespacial completo. La elasticidad del hit rate respecto al sesgo Zipf indica que vale la pena segmentar por zona/tipo de consulta al dimensionar el caché.

¿Cuál política recomendar? En el régimen experimentado (sin evicciones), las tres son estadísticamente equivalentes (diferencias <1 pp en hit rate y <2 qps en throughput). Bajo presión real de memoria: LRU para cargas con localidad temporal; LFU para patrones Zipf estables y corridas largas (≥5 min); FIFO sólo por simplicidad. Para el caso de uso de logística, LRU es la elección más robusta.

¿Cómo afecta el TTL? En corridas cortas (25 s) ningún TTL expira y el efecto observado es nulo. En la corrida larga (180 s) el TTL actúa como techo del hit rate por consulta: Q1 con TTL = 300 s alcanza 97 %, mientras que Q4 con TTL = 120 s se queda en 62 % en el mismo régimen. Esto sugiere que TTLs muy cortos son útiles cuando el dato real cambia rápido, pero penalizan el hit rate; TTLs largos maximizan la reutilización a costa de coherencia.

7. Despliegue y reproducibilidad

El sistema se despliega completamente con:

```
docker compose up -d --build              # 4 servicios + Redis
python experiments/master_run.py          # 22 corridas (~15 min)
python experiments/build_figures.py       # regenera figuras PDF/PNG
```

El archivo .env controla la política (CACHE_POLICY), el tamaño (REDIS_MAXMEMORY) y los TTLs por consulta. El script master_run.py ejecuta automáticamente las 22 corridas (3 políticas × 3 tamaños × 2 distribuciones + 3 caché pequeño + 1 larga). El código completo está en el repositorio adjunto con README.md de despliegue y docker-compose.yml.

7.1. Guion del video de demostración (10 min)

1. (0:00-1:00) Contexto del problema y arquitectura general (Figura 2 del enunciado).
2. (1:00-2:30) docker compose up -d -build; revisión de healthchecks en los puertos 8000-8003.
3. (2:30-4:00) Consultas Q1-Q5 manuales con curl al puerto 8001 mostrando primero MISS y luego HIT en la misma key.
4. (4:00-5:30) python experiments/master_run.py -suite demo: un experimento Zipf de 15 s, viendo /status y /summary.
5. (5:30-7:00) Comparación en vivo Zipf vs. Uniforme y lectura del hit rate por consulta (/summary/by_query).
6. (7:00-8:30) Cambio de política en runtime con redis-cli CONFIG SET maxmemory-policy allkeys-lfu y nuevo experimento.
7. (8:30-10:00) Apertura de las figuras generadas y discusión breve de los hallazgos (Cuadro 8).

8. Conclusiones

Se implementó una plataforma distribuida funcional con caché configurable sobre Redis y se evaluaron empíricamente cuatro hipótesis:

1. La distribución Zipf casi triplica el hit rate respecto a Uniforme (∼80 % vs. ∼27 %), confirmando que la concentración de consultas en items calientes es el factor dominante del rendimiento de la caché. El throughput sostenido se diferencia aún más (38 vs. 10 qps, factor ∼4) porque los misses saturan el Response Generator.
2. Los tres tamaños de caché (50/200/500 MB) son equivalentes para este keyspace de ∼300 KB observados; ningún experimento (ni siquiera el de 2 MB) registró evicciones. La diferencia entre LRU/LFU/FIFO es ruido estadístico (<1 pp). Una corrida adicional de saturación deliberada (LRU 1.2 MB Uniforme 60 s) sí produjo 475 evicciones (rate 408/min) y un hit rate de 19 %, validando la lógica de evicción del sistema (Cuadro 4).

3. La caché reduce la latencia 3-4 órdenes de magnitud: p50 hit ∼0.3 ms vs. p50 miss ∼870 ms; cache efficiency 1 008-1 103 ms/consulta en Zipf 25 s.
4. Los TTLs operan como techo en estado estacionario: la corrida larga (180 s, LRU, Zipf) alcanza un hit rate global de 93 % y exhibe el efecto del TTL por consulta (Q1 97 %, Q4 62 %). Las corridas cortas no alcanzan a expirar claves.

Las siguientes entregas implementarán colas de procesamiento y fallback (Entrega 2) y un dashboard de visualización (Entrega 3), reutilizando los servicios y el sistema de métricas desarrollado aquí.